데이터 병합, 학습 제외 컬럼과 모델 평가 지표

머신러닝 모델의 성능은 모델 선택만으로 결정되지 않는다. 여러 모델의 결과를 하나의 표로 합쳐 비교하고, 예측 시점에 알 수 없는 정보나 의미가 없는 컬럼을 제외하며, 문제 유형에 맞는 평가 지표를 선택해야 한다.

이 글은 자전거 공유 수요 예측, 주택 임대료 예측, 호텔 예약 취소 예제에서 사용한 코드를 바탕으로 다음 세 가지를 정리한다.

1. `pd.concat()`으로 모델 결과를 병합하는 방법
2. 모델 학습에서 제외할 컬럼을 판단하고 제거하는 방법
3. 회귀와 분류 모델의 평가 지표

---

1. `pd.concat()`으로 결과 데이터 병합하기

여러 모델을 학습하면 각 모델의 평가 결과를 하나의 DataFrame으로 모아 비교할 필요가 있다. 자전거 공유 수요 예제에서는 기존 모델 결과와 튜닝한 결정 트리의 결과를 다음처럼 합쳤다.

```
final_result_df = pd.concat(
    [result_df, pd.DataFrame([tuned_dt_result])],
    ignore_index=True
)

final_result_df = final_result_df.sort_values('RMSE').reset_index(drop=True)
```

코드 흐름

코드	역할
`result_df`	기존 모델들의 평가 결과가 담긴 DataFrame이다.
`tuned_dt_result`	튜닝한 결정 트리의 모델명과 평가 지표가 담긴 딕셔너리다.
`pd.DataFrame([tuned_dt_result])`	딕셔너리를 한 행짜리 DataFrame으로 바꾼다.
`pd.concat([...])`	DataFrame들을 행 방향으로 이어 붙인다.
`ignore_index=True`	합친 뒤 인덱스를 `0`부터 다시 부여한다.
`sort_values('RMSE')`	RMSE가 낮은 모델이 위에 오도록 정렬한다.
`reset_index(drop=True)`	정렬 전 인덱스를 버리고 새 인덱스를 만든다.

병합 전후 예시

```
import pandas as pd

result_df = pd.DataFrame([
    {'model': 'LinearRegression', 'RMSE': 410.2},
    {'model': 'RandomForest', 'RMSE': 290.5},
])

tuned_dt_result = {
    'model': 'TunedDecisionTree',
    'RMSE': 305.8,
```

```
}

final_result_df = pd.concat(
    [result_df, pd.DataFrame([tuned_dt_result])],
    ignore_index=True,
)

final_result_df = final_result_df.sort_values('RMSE').reset_index(drop=True)
print(final_result_df)
```

출력:

```
                model   RMSE
0       RandomForest  290.5
1  TunedDecisionTree  305.8
2   LinearRegression 410.2
```

`pd.concat()`은 같은 형식의 결과표를 **위아래로 추가**할 때 적합하다. 기본값은 `axis=0`이므로 행 방향 병합이다. 열 이름이 서로 다르면 없는 값은 `NaN`으로 채워질 수 있다.

> 이 예제의 `concat`은 NumPy 배열을 연결하는 `np.concatenate()`가 아니라 Pandas DataFrame을 연결하는 `pd.concat()`이다.

---

2. 모델 학습에서 제외할 컬럼

컬럼을 제거하는 목적은 단순히 데이터 크기를 줄이는 데 있지 않다. 정답을 미리 알려주는 정보, 예측 시점에 알 수 없는 정보, 모든 행에서 같은 값을 가진 정보, 목표값 자체를 입력값에서 제거하기 위해 사용한다.

2.1 목표값을 입력값에서 분리하기

주택 임대료 예측에서는 `Rent`가 맞혀야 할 목표값이다. 목표값을 입력 특성에 그대로 포함하면 모델이 답을 보고 학습하는 문제가 생긴다.

```
X = rent_df.drop('Rent', axis=1)
y = rent_df['Rent']
```

변수	의미
`X`	모델이 입력으로 받는 독립변수, 즉 특성이다.
`y`	모델이 예측해야 하는 종속변수, 즉 목표값이다.

`drop('Rent', axis=1)`은 `Rent` 열을 제외한 나머지 열을 반환한다. 원본 `rent_df`는 바뀌지 않는다.

2.2 원본 표현을 정리하기

주택 임대료 예제에는 `Floor`, `Posted On`의 원본 컬럼을 제거하는 코드가 있다.

```
rent_df.drop(['Floor', 'Posted On'], axis=1, inplace=True)
```

- `axis=1`은 행이 아닌 열을 대상으로 한다.
- `inplace=True`는 반환값을 새 변수에 저장하지 않고 원본 DataFrame을 직접 바꾼다.
- 원본 텍스트나 날짜 컬럼을 제거하기 전에, 그 안에서 필요한 값을 별도 특성으로 만들었는지 확인해야 한다.

예를 들어 `Posted On`에서 연도, 월, 요일을 추출했다면 원본 날짜 문자열 자체는 학습에 필요하지 않을 수 있다. 반대로 아직 필요한 정보를 추출하지 않았다면 성급히 제거하면 안 된다.

2.3 데이터 누수 가능성이 있는 컬럼 제거하기

호텔 예약 취소 예제에서는 실제 예약 시점에 알기 어렵거나, 취소 여부를 직접 드러낼 수 있는 컬럼을 먼저 제외했다.

```
leakage_or_post_booking_cols = [
    'reservation_status',
    'reservation_status_date',
    'assigned_room_type',
]

redundant_cols = [
    'adults', 'children', 'babies',
    'stays_in_weekend_nights', 'stays_in_week_nights',
    'arrival_date_month',
    'previous_cancellations',
    'previous_bookings_not_canceled',
]

cols_to_drop = [
    col
    for col in leakage_or_post_booking_cols + redundant_cols
    if col in hotel.columns
]

hotel = hotel.drop(columns=cols_to_drop)
```

`reservation_status`는 취소 여부와 직접 연결된 정보이므로 입력으로 쓰면 안 된다. 이런 정보가 학습 데이터에 들어가면 테스트 성능은 비정상적으로 좋아 보이지만, 실제 서비스에서는 사용할 수 없는 모델이 된다. 이를 **데이터 누수(Data Leakage)** 라고 한다.

`if col in hotel.columns` 조건은 데이터셋 버전에 따라 특정 컬럼이 없어도 `KeyError` 없이 코드를 실행하도록 돕는다.

2.4 값이 하나로 고정된 컬럼 제거하기

자전거 공유 수요 예제는 운영일만 남긴 뒤 `Functioning Day`를 제거했다.

```
model_df = bike_df[bike_df['Functioning Day'] == 'Yes'].copy()
model_df = model_df.drop(columns=['Functioning Day'])
```

필터링 이후 남은 모든 행에서 `Functioning Day` 값은 `Yes`다. 값이 변하지 않는 상수 컬럼은 행을 구분하는 데 도움이 되지 않으므로 모델 입력에서 제외할 수 있다.

컬럼 제거 전 점검표

확인할 질문	제거를 고려하는 경우
예측 시점에 이 값을 알 수 있는가?	알 수 없다면 데이터 누수 가능성이 있다.
목표값이나 결과를 직접 알려주는가?	그렇다면 입력값에서 제거해야 한다.
모든 행의 값이 같은가?	상수 컬럼이라면 정보량이 없다.
다른 컬럼으로 필요한 정보를 만들었는가?	원본 컬럼을 제거하기 전에 파생 특성을 확인한다.
도메인상 실제로 불필요한가?	단순한 추측이 아니라 근거를 확인한다.

3. 모델 평가 지표

평가 지표는 문제 유형에 맞게 골라야 한다. 숫자를 예측하는 회귀 문제와 클래스를 맞히는 분류 문제는 오차를 해석하는 방식이 다르다.

3.1 회귀 평가 지표

주택 임대료와 자전거 수요 예제는 연속된 숫자를 예측하므로 회귀 지표를 사용한다.

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    root_mean_squared_error,
    r2_score,
)

mae = mean_absolute_error(y_test, pred)
mse = mean_squared_error(y_test, pred)
rmse = root_mean_squared_error(y_test, pred)
r2 = r2_score(y_test, pred)
```

지표	의미	좋은 방향
MAE	실제값과 예측값의 절대 오차 평균	낮을수록 좋다.
MSE	오차를 제곱한 뒤 평균낸 값	낮을수록 좋다. 큰 오차에 더 민감하다.
RMSE	MSE에 제곱근을 적용한 값	낮을수록 좋다. 목표값과 같은 단위로 해석하기 쉽다.
R^2	모델이 목표값의 변화를 설명하는 정도	1에 가까울수록 좋다. 0 이하면 평균 예측보다 나쁠 수 있다.

예를 들어 자전거 대여 건수를 예측할 때 RMSE가 `120`이면, 큰 오차를 더 크게 반영했을 때의 대표 오차 크기가 약 120대 수준이라는 뜻으로 해석할 수 있다. MAE와 RMSE를 함께 보면 큰 오차가 유난히 많은지도 파악할 수 있다.

3.2 분류 평가 지표

호텔 예약 취소 예제는 취소 여부인 `0` 또는 `1`을 맞히는 이진 분류 문제다.

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
)

accuracy = accuracy_score(y_test, pred)
precision = precision_score(y_test, pred, zero_division=0)
recall = recall_score(y_test, pred, zero_division=0)
f1 = f1_score(y_test, pred, zero_division=0)
auc = roc_auc_score(y_test, proba)
```

지표	의미	언제 특히 중요한가
Accuracy	전체 예측 중 맞힌 비율	클래스 비율이 비교적 균형일 때 기본 지표로 사용한다.
Precision	취소라고 예측한 것 중 실제 취소인 비율	잘못된 취소 예측으로 불필요한 대응을 줄이고 싶을 때 본다.
Recall	실제 취소 중 모델이 취소로 찾아낸 비율	취소 고객을 최대한 놓치지 않는 것이 중요할 때 본다.
F1-score	Precision과 Recall의 조화 평균	두 지표를 균형 있게 보고 싶을 때 사용한다.
ROC-AUC	여러 임계값에서 양성과 음성을 구분하는 능력	확률 예측 모델의 전반적 분류 능력을 비교할 때 사용한다.

`zero_division=0`은 Precision이나 Recall을 계산할 때 분모가 0인 상황에서 경고 또는 오류 대신 `0`을 반환하도록 설정한다.

3.3 혼동 행렬과 분류 보고서

정확도 하나만 보면 어떤 종류의 실수를 했는지 알기 어렵다. 혼동 행렬은 실제 클래스와 예측 클래스의 조합을 보여준다.

```
from sklearn.metrics import confusion_matrix, classification_report

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))
```

이진 분류에서 주요 값은 다음과 같다.

값	의미
TP	실제 양성을 양성으로 맞힌 수
TN	실제 음성을 음성으로 맞힌 수
FP	실제 음성을 양성으로 잘못 예측한 수
FN	실제 양성을 음성으로 놓친 수

손글씨 도형 분류와 AlexNet 예제도 혼동 행렬과 `classification_report()`를 사용한다. 다중 분류에서는 클래스별 Precision, Recall, F1-score를 따로 확인해야 특정 클래스만 반복해서 틀리는 문제를 발견할 수 있다.

3.4 손실 함수와 평가 지표의 차이

딥러닝 학습 코드에는 다음처럼 `MSELoss`, `CrossEntropyLoss` 같은 손실 함수가 나온다.

```
loss = loss_fn(y_pred, y_train)
loss.backward()
optimizer.step()
```

손실 함수는 역전파로 가중치를 업데이트하기 위한 학습 기준이다. 반면 Accuracy, F1-score, RMSE 같은 평가 지표는 학습된 모델의 성능을 사람이 해석하고 모델을 비교하기 위한 기준이다. 둘은 비슷한 숫자가 아니라 역할이 다르다.

정리

- `pd.concat()`은 여러 모델의 평가 결과를 하나의 DataFrame으로 모을 때 유용하다.
- 목표값, 데이터 누수 가능성이 있는 값, 상수 컬럼, 의미를 잃은 원본 컬럼은 입력 특성에서 제외할 수 있다.
- 회귀에서는 MAE, MSE, RMSE, R^2를 사용한다.

- 분류에서는 Accuracy만 보지 말고 Precision, Recall, F1-score, ROC-AUC, 혼동 행렬을 함께 확인한다.
- 손실 함수는 학습을 위한 기준이고, 평가 지표는 성능 해석과 비교를 위한 기준이다.